

SmartLibrary Taller práctico Bloque 5

Josué David Guerrero Vélez

De clases aisladas a objetos que colaboran

El diseño distingue las relaciones entre usuarios, libros, ejemplares y préstamos. La implementación valida la renovación y conserva su historial. Los diagramas y el código están en el repositorio público de la entrega.

<https://github.com/DvklIz/SmartLibrary-Bloque5>

1 Diagnóstico de relaciones

Elementos	Significado e independencia	Decisión inicial
Estudiante y Prestamo	El estudiante solicita préstamos. Puede estar registrado sin tener uno; el registro de préstamo conserva la referencia al estudiante.	Asociación
Libro y Ejemplar	El libro reúne las copias de un título. La copia física puede conservarse aunque se retire la ficha del catálogo.	Agregación con supuesto
Prestamo y Renovacion	La renovación es una entrada del historial de un préstamo. Sin ese préstamo no tiene significado propio.	Composición
Usuario y Estudiante	El estudiante es un usuario de la biblioteca con código estudiantil y programa. No son dos objetos que formen un todo.	Generalización
Usuario y Bibliotecario	El bibliotecario también es un usuario de la biblioteca, con código de empleado y turno.	Generalización

Preguntas de discusión

Relacionar dos clases no significa que una sea parte de la otra. Un estudiante utiliza un ejemplar mediante un préstamo, pero no es dueño de su ciclo de vida.

R10 establece un único libro por ejemplar; R11 define las renovaciones de cada préstamo; R12 identifica a estudiantes y bibliotecarios como usuarios. El ciclo de vida de los ejemplares no está definido por R10, por eso se declara un supuesto.

Libro–Ejemplar y Prestamo–Renovacion no se pueden decidir solo por sus nombres. También la herencia necesita comprobar que las especializaciones sean tipos reales de usuario.

2 Relaciones estructurales

Relación	Tipo y multiplicidad	Justificación
Estudiante–Préstamo	Asociación 1 estudiante por préstamo; 0..* préstamos por estudiante.	El préstamo conoce a su solicitante. Registrar o terminar un préstamo no crea ni elimina al estudiante.
Préstamo–Ejemplar	Asociación 1 ejemplar por préstamo; 0..* préstamos históricos por ejemplar.	El ejemplar sigue existiendo después de devolverlo. La multiplicidad cuenta el historial, no préstamos activos simultáneos.
Libro–Ejemplar	Agregación 1 libro por ejemplar; 0..* ejemplares por libro.	Un título agrupa copias físicas con identidad propia. Se admite una ficha sin copias disponibles o registradas.
Préstamo–Renovación	Composición 1 préstamo por renovación; 0..* renovaciones por préstamo.	El préstamo crea y conserva su historial. Una renovación no se comparte con otro préstamo ni se registra por separado.

Supuesto para R10: retirar una ficha del catálogo no destruye las copias físicas. Antes de eliminarla del registro, sus ejemplares se reasignarían a una ficha del mismo libro o se retirarían del inventario. Mientras un ejemplar esté registrado, mantiene un único Libro. Con una regla diferente de ciclo de vida habría que revisar la agregación.

3 Generalización de Usuario

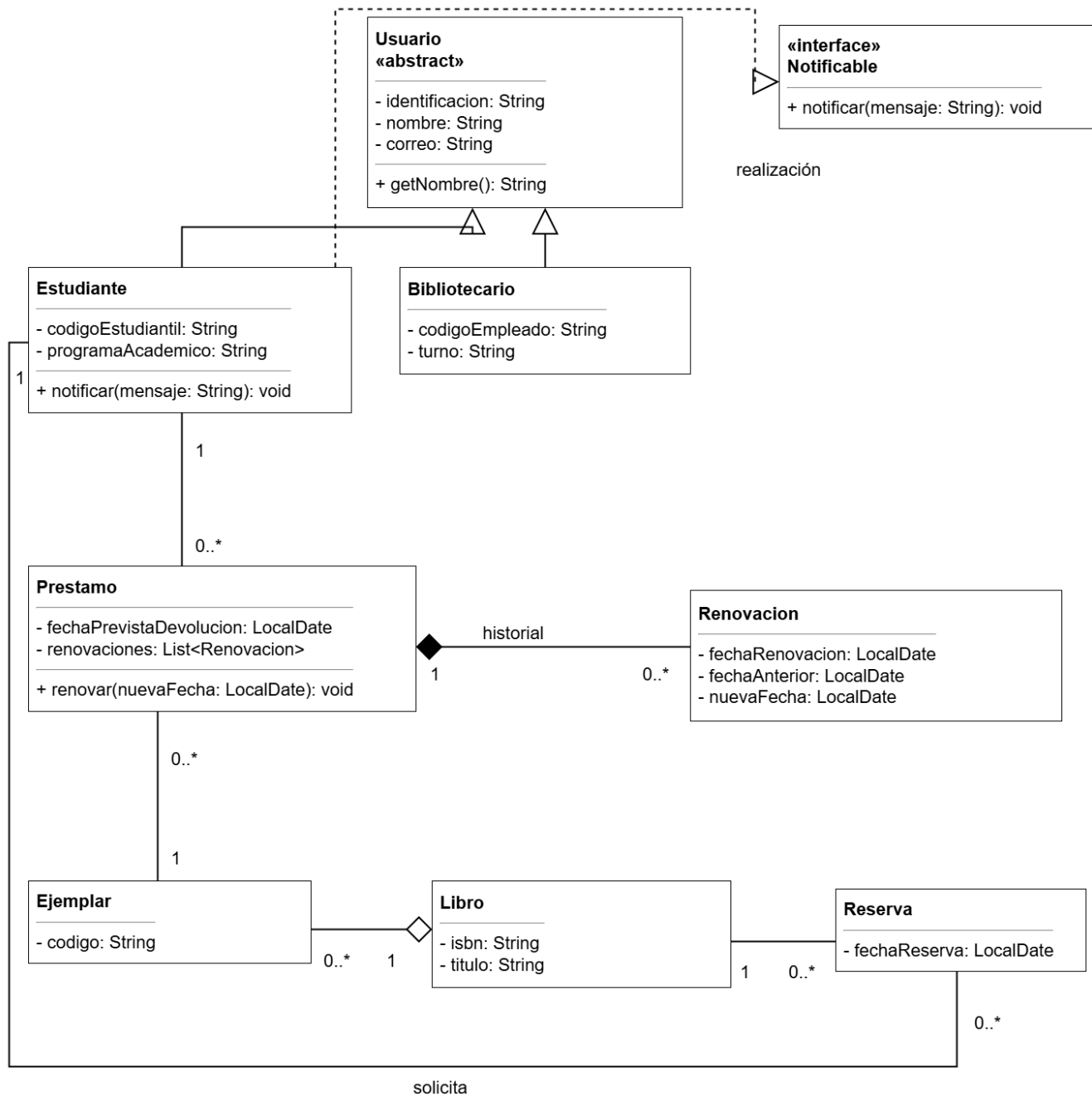
Estudiante es un Usuario porque se identifica y participa en la biblioteca como solicitante de préstamos y reservas. Bibliotecario también es un Usuario, aunque participa desde su función de atención. Ambos pueden tratarse como usuarios para consultar su nombre e identificación; sus datos particulares quedan en cada especialización.

Usuario se define abstracta porque, en este fragmento, las personas se crean como Estudiante o Bibliotecario. La herencia se justifica por R12 y por esa relación «es-un», no solo por ahorrar tres atributos.

Por ejemplo, una editorial podría tener identificación, nombre y correo. Hacerla heredar de Usuario solo para reutilizar esos campos confundiría a una organización del catálogo con una persona que usa la biblioteca. Compartir datos no basta para pertenecer a la misma jerarquía.

4 Diagrama de clases actualizado

SmartLibrary Diagrama de clases



El triángulo apunta a Usuario o Notificable. La realizaci3n se dibuja con l3nea discontinua; los rombos quedan en Libro y Prestamo. Las asociaciones de Reserva completan la vista del dominio: cada reserva pertenece a un estudiante y solicita un libro. Reserva se conserva solo en el modelo de este taller.

[Diagrama de clases editable en Visual Paradigm Online](#)

5 Contrato de notificación

Estudiante implementa Notificable para recibir el aviso de renovación. Bibliotecario no lo implementa en este fragmento: los requisitos no exigen avisos para su función. Es una decisión de alcance para R13, no una restricción permanente sobre los bibliotecarios.

```
public interface Notificable {  
    void notificar(String mensaje);  
}
```

Pregunta	Respuesta
¿Qué promete una clase que implementa Notificable?	Ofrecer el método público void notificar(String mensaje). Quien recibe un Notificable puede invocarlo.
¿La interfaz define cómo se envía el mensaje?	No. No fija un canal, formato, almacenamiento ni entrega efectiva. En esta prueba se simula con una línea en consola.
¿Puede heredar de Usuario e implementar Notificable?	Sí. Estudiante extends Usuario implements Notificable: la clase base representa qué tipo de objeto es; la interfaz expresa una capacidad.

Una interfaz no equivale a una superclase. Usuario aporta los atributos comunes y el constructor de la jerarquía. Notificable aporta la firma de un comportamiento, sin estado ni constructor de usuario.

6 Implementación representativa

El repositorio contiene Usuario, Estudiante, Bibliotecario, Renovacion y Prestamo. Libro y Ejemplar permiten construir el préstamo de la prueba; PruebaSmartLibrary ejecuta los casos. Los constructores de las especializaciones llaman a super y guardan sus datos particulares. SmartLibraryInterfaz permite ejecutar el mismo ejemplo desde una ventana, sin ampliar las funciones del dominio.

```
@Override  
public void notificar(String mensaje) {  
    System.out.println("Notificacion para "  
        + getNombre() + ": " + mensaje);  
}
```

Renovacion conserva fechaRenovacion, fechaAnterior y nuevaFecha como LocalDate, con getters y sin métodos para cambiarlas. Su constructor tiene acceso de paquete. Prestamo crea las entradas, mantiene la lista privada y entrega una vista de solo lectura para consultar el historial.

7 Renovación y evidencia de ejecución

La validación ocurre antes de crear o guardar la renovación. Si la fecha no es posterior a la vigente, se lanza `IllegalArgumentException` y el préstamo conserva su estado.

```
public void renovar(LocalDate nuevaFecha) {
    if (nuevaFecha == null ||
        !nuevaFecha.isAfter(fechaPrevistaDevolucion)) {
        throw new IllegalArgumentException(
            "La nueva fecha debe ser posterior a la fecha prevista vigente.");
    }
    Renovacion renovacion = new Renovacion(
        LocalDate.now(), fechaPrevistaDevolucion, nuevaFecha);
    renovaciones.add(renovacion);
    fechaPrevistaDevolucion = nuevaFecha;
}
```

Se compiló el fragmento con JDK 25 para Java 17 y se ejecutó `PruebaSmartLibrary` el 30 de septiembre de 2026. La salida completa está en `evidencia/ejecucion.txt`. Las comprobaciones generan un error si no se conserva alguna de las fechas o si un rechazo altera el historial.

Caso	Resultado observado
Renovación válida	Fecha prevista inicial: 2026-10-08. Nueva fecha: 2026-10-15. Fecha de renovación: 2026-09-30. Historial: 1 entrada.
Fecha igual	Intento con 2026-10-15: rechazado con <code>IllegalArgumentException</code> .
Fecha anterior	Intento con 2026-10-14: rechazado con <code>IllegalArgumentException</code> .
Estado tras los rechazos	Fecha prevista: 2026-10-15. Historial: 1 entrada. Resultado: pruebas correctas.

Notificacion para Ana: Su prestamo fue renovado.
Fecha anterior: 2026-10-08
Fecha de renovacion: 2026-09-30
Nueva fecha: 2026-10-15
Cantidad de renovaciones: 1
Fecha despues de los rechazos: 2026-10-15
Renovaciones despues de los rechazos: 1
Resultado: pruebas correctas.

Prueba desde la interfaz

La ventana ejecuta la renovación del mismo préstamo. Se abre con Ejecutar_SmartLibrary.cmd o con SmartLibrary.jar. La fecha vigente y la tabla permiten comprobar el resultado de cada intento.

SmartLibrary — Renovación de préstamos

SmartLibrary

Renovación de un préstamo: Taller Bloque 5

Estudiante: Ana | Libro: Introducción a Java | Ejemplar: E301

Fecha prevista de devolución: 2026-10-15

Renovaciones registradas: 1

Nueva devolución (AAAA MM DD): 2026 10 15

Fecha de renovación	Fecha anterior	Nueva fecha
2026 09 30	2026 10 08	2026 10 15

Resultado y notificación

Ejemplo listo. Renueva al 15 de octubre y luego intenta la misma fecha o una anterior.
Renovación registrada: 2026-10-15.
Notificación para Ana: Su préstamo fue renovado.

Renovación válida del 8 al 15 de octubre de 2026. El historial registra las tres fechas y queda con una entrada.

SmartLibrary — Renovación de préstamos

SmartLibrary

Renovación de un préstamo: Taller Bloque 5

Estudiante: Ana | Libro: Introducción a Java | Ejemplar: E301

Fecha prevista de devolución: 2026-10-15

Renovaciones registradas: 1

Nueva devolución (AAAA MM DD): 2026 10 15

Fecha de renovación	Fecha anterior	Nueva fecha
2026 09 30	2026 10 08	2026 10 15

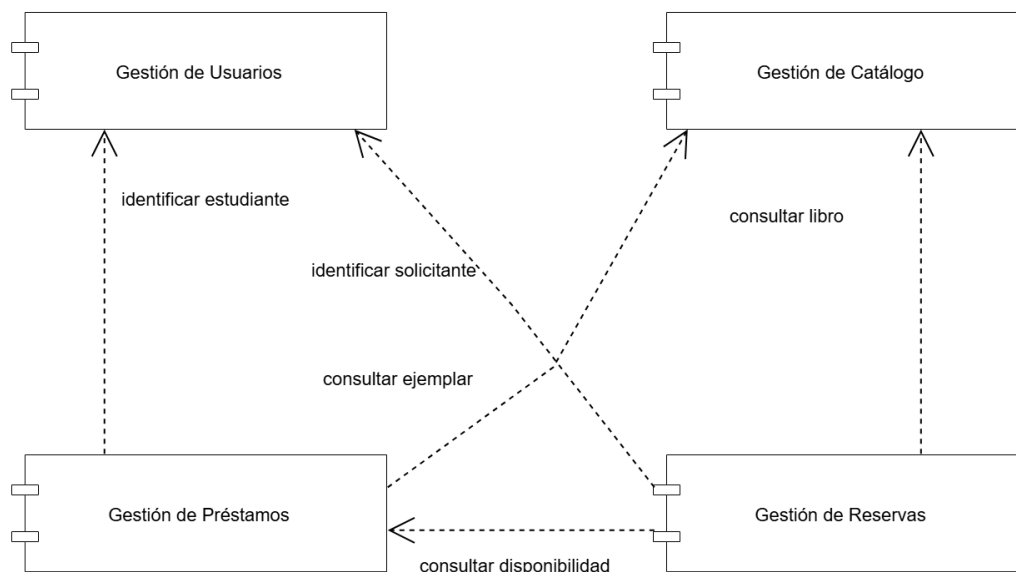
Resultado y notificación

Ejemplo listo. Renueva al 15 de octubre y luego intenta la misma fecha o una anterior.
Renovación registrada: 2026-10-15.
Notificación para Ana: Su préstamo fue renovado.
Renovación rechazada: La nueva fecha debe ser posterior a la fecha prevista vigente.
La fecha vigente y el historial se conservan.

Intento con la misma fecha vigente. Se rechaza la renovación y se conserva la fecha del 15 de octubre y una sola entrada en el historial.

8 Vista funcional mediante componentes UML

SmartLibrary Diagrama de componentes



Las flechas discontinuas apuntan al componente del que se necesita información. Los cuatro componentes agrupan funciones de SmartLibrary, de acuerdo con R14.

[Diagrama de componentes editable en Visual Paradigm Online](#)

Componente	Clases principales	Colaboración necesaria
Gestión de Usuarios	Usuario, Estudiante, Bibliotecario y contrato Notificable.	Mantiene la información de usuarios que consultan Préstamos y Reservas. No necesita otro componente para este fragmento.
Gestión de Catálogo	Libro y Ejemplar.	Mantiene títulos y copias que consultan Préstamos y Reservas. No necesita otro componente para este fragmento.
Gestión de Préstamos	Prestamo y Renovacion.	Consulta Usuarios para identificar al estudiante y Catálogo para identificar el ejemplar prestado.
Gestión de Reservas	Reserva.	Consulta Usuarios para identificar al solicitante, Catálogo para conocer el libro y Préstamos para conocer la ocupación de sus ejemplares.

9 Conclusión

Al diseñar las relaciones, ya no basta con listar los datos de cada clase. Hay que decidir quién conoce a quién y qué objetos dependen del ciclo de vida de otros. En SmartLibrary, el estudiante y el ejemplar mantienen su identidad después de un préstamo, mientras que una renovación solo tiene sentido dentro de su historial. La herencia permite reconocer a estudiantes y bibliotecarios como usuarios, pero necesita una razón del dominio.

Notificable añade una capacidad sin obligar a todos los usuarios a tenerla. Estas decisiones también afectan al código: renovar debe conservar las fechas anteriores y rechazar cambios inválidos. La vista de componentes ayuda a ubicar las funciones y sus colaboraciones. El resultado es un modelo que explica las reglas de la biblioteca y un fragmento de código que permite comprobarlas.